Urban Mobility Data Explorer: Technical Report

Team: TODO team name | GitHub: TODO link | Video walkthrough: TODO link | NYC yellow taxi trips, January 2019

1. Problem framing and dataset analysis

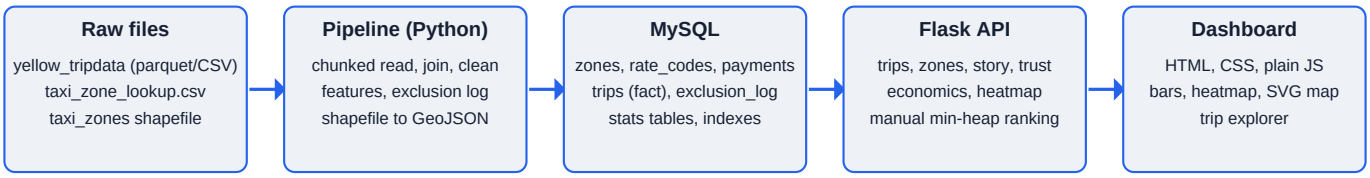
We worked with the TLC yellow-taxi file for January 2019 (**7,667,792 trips**: pickup and dropoff times, distance, rate code, payment type and itemised fares), joined to *taxi_zone_lookup* (zone id to borough, zone and service zone) and the *taxi_zones* shapefile (263 polygons for 260 ids, converted to lat/lon GeoJSON). Instead of building a general browser for the data we asked one question: **what does slow traffic cost a taxi rider, and how far can the numbers be trusted?** Every view of the dashboard carries the number of trips behind it and the number of records removed.

Data challenge	Trips	Decision
Distance of 0	54,770	Excluded: minutes per mile cannot be computed
Trip longer than 3 hours (meter left running)	20,913	Excluded
Negative fare (refund or reversal)	7,131	Excluded; the 406 rate-code-2 trips at exactly -$52 are all refunds
Speed above 80 mph, zero duration, dropoff before pickup	6,135 / 6,290 / 4	Excluded
Pickup date outside January 2019 (years 2001 to 2088)	537	Excluded
Unknown pickup or dropoff zone (264, 265)	170,637	Kept and flagged; never ranked or mapped
Passenger count 0 or above 6; speed below 2 mph	115,104 / 11,585	Kept and flagged

Assumptions. A metered yellow-cab trip cannot exceed 80 mph, 3 hours, 100 miles or $500; all thresholds are constants in *pipeline/config.py*. A fare of zero or less is a reversal, not a trip. Trips in zones 264 and 265 are real trips with an unknown place, so we keep and flag them. *congestion_surcharge* is empty for most rows (it only started in 2019), so empty means 0. Cash tips are not recorded (311 of 2,137,415 cash trips show a tip, against 5,257,023 of 5,486,027 card trips), so we build no tip feature on cash. In total 88,946 trips (1.16%) were removed; every removal is written to *exclusion_log* with a reason code (98,524 log rows, because one trip can break several rules).

Unexpected observation. 537 trips are dated between 2001 and 2088 inside a January 2019 file, and impossible speeds sit in the raw averages. Cleaning moves the mean speed from 13.17 mph (raw) to 11.72 mph (cleaned), an 11% change, so a cleaning decision changes the headline number. This is why we added a raw-versus-cleaned toggle and a data-trust section to the dashboard instead of hiding the cleaning.

2. System architecture and design decisions



data/processed (parquet, zones.geojson, stats CSVs) sits between the pipeline and MySQL; the dashboard only calls the API, never the database.

Stack. The pipeline is Python with pandas, reading the file in chunks of 500,000 rows so 7.7 million rows fit in memory, and keeping column types in a parquet file between stages. MySQL stores the result because the data is relational (trips refer to zones, rate codes and payment types) and because the dashboard needs filtered, sorted, paged queries. Flask serves read-only JSON; the credentials come from environment variables, never from the code. The frontend is plain HTML, CSS and JavaScript with no library, so the whole team can read and explain every line.

Schema. *trips* is the fact table. *zones* is a dimension keyed by *location_id*, because the trip file already stores PULocationID and DOLocationID with exactly those numbers. *borough* stays a column of *zones* instead of its own table: there are only seven values and a table would add a join without adding information. *payment_types* (codes 1 to 6) and *rate_codes* (including 99) are lookup tables, and we keep every code so that a foreign key does not fail on data from another month. *exclusion_log* points to *exclusion_reasons* but not to *trips*, because an excluded row is never loaded. Four statistics tables (*zone_hour_stats*, *hour_stats*, *daily_stats*, *econ_hour*) hold precomputed medians and averages. Indexes follow the queries: pickup time, pickup zone plus time, pickup and dropoff zone (top routes), date plus hour, and fare and distance for sorting. They are created *after* the bulk load, because each index slows every insert.

Trade-offs. (1) The derived columns (speed, minutes per mile, fare per minute, hour, weekend flag) are stored in *trips*, which duplicates information but avoids recomputing them for millions of rows on every request. (2) The dashboard reads precomputed medians, because MySQL has no median function and grouping 7.5 million rows on every click is too slow; the price is that the statistics tables must be rebuilt when the cleaning rules change. (3) Paging with *OFFSET* and a total *COUNT(*)* is simple but slow on the full table, so we measure it on the loaded database and keep a sample load as a fallback. (4) We indexed fare and distance but not duration and speed, to save space. (5) The first call of the time heatmap groups the whole table and then caches the answer in memory.

3. Algorithmic logic and data structures

Problem. The dashboard needs the K slowest pickup zones and the K busiest routes. We wrote a **min-heap** by hand in *backend/algorithms/custom_algorithm.py* (no *heapq*, no *sorted*, no *ORDER BY* for the ranking). SQL only aggregates (one row per zone, or per pickup-dropoff pair, with at least 30 trips); the ranking is done by the heap. The heap is a flat list where the parent of index *i* is *(i-1)//2* and its children are *2i+1* and *2i+2*. Every parent is smaller than or equal to its children, so the root is always the minimum, and popping K times returns the K smallest in increasing order. For "largest first" (busiest routes) we push the negative value.

```
push(item)                                      pop()
 append item at the end; i = last index          if heap empty: return nothing
 while i > 0:                                    root = items[0]; last = items.removeLast()
   p = (i - 1) // 2                              if heap now empty: return root
   if key(items[i]) < key(items[p]):             items[0] = last; i = 0
       swap(items[i], items[p]); i = p            loop:
   else: stop                                       l = 2i+1; r = 2i+2; if l >= n: stop
                                                    c = l; if r < n and key(items[r]) < key(items[l]): c = r
k_smallest(items, k, key):                          if key(items[c]) < key(items[i]): swap; i = c
 for item in items: if key(item) is not None:       else: stop
     push((key(item), item))                    return root
 repeat k times: pop and collect
```

Operation	Time	Reason
push / pop	O(log n)	the item moves at most one level per step along a path of height floor(log2 n)
build from n pushes + K pops	O(n log n + K log n)	n pushes, then K pops
space	O(n)	one list holding the n (key, item) pairs

Why not just sort? For about 260 zones a sort is as fast and the heap is not needed for speed; we use it to rank real data with our own code. It matters more for routes, where many thousands of pairs are ranked. The heap compares only the key, so equal keys come out in no defined order. **Improvement not yet done:** keeping a heap of size K (replace the root when a better item arrives) would cost O(n log K) time and O(K) space instead of O(n log n) and O(n). Correctness is tested against Python's own sorting in *tests/test_algorithms.py*.

4. Insights and interpretation

INSERT SCREENSHOT: hourly bars (minutes per mile, speed, fare per minute), weekdays

INSERT SCREENSHOT: slowness map and slowest-zones table, 8 to 10 AM weekdays

INSERT SCREENSHOT: raw versus cleaned mean speed by hour

Insight 1: slow hours cost riders more than twice the time per mile. From *hour_stats* (medians, via */api/story/hour-profile?day_type=weekday*). On weekdays the median trip takes 7.6 minutes per mile at 09:00 and 3.5 at 04:00. At 09:00 the median fare per mile is $6.49 against $4.00 at 04:00, while the fare per minute falls from $1.19 to $0.82. *Meaning:* in traffic the rider pays about 62% more per mile and the driver earns about 31% less per minute, because the meter charges for waiting time at a lower rate than for moving. Congestion has a price on both sides of the cab.

Insight 2: the slowest pickup zones are concentrated in central Manhattan. Ranking weekday 08:00 to 09:59 pickups with the manual heap (at least 30 trips, unknown zones removed; */api/story/slowest-zones?hour_from=8&hour_to=9&day_type=weekday*) gives Sutton Place/Turtle Bay North (229), Penn Station/Madison Sq West (186), Midtown Center (161), Midtown East (162) and Garment District (100), at about 8.7 to 9.4 minutes per mile. *Meaning:* the morning bottleneck is a small area around Penn Station and Midtown, not the whole borough, which is where a congestion measure or a bus lane would have the most effect. Each zone is shown with its trip count so a small sample cannot top the list.

Insight 3: the cleaning rules change the answer, most of all at night. From */api/trust/raw-vs-clean*: raw mean speed is 13.17 mph and cleaned mean speed is 11.72 mph. The gap is largest at 01:00 on weekdays (4.2 mph), where few trips exist and a few impossible speeds weigh the most. *Meaning:* any city report built on the raw file would overstate speeds, and most of all in the hours with the least traffic. 57.7% of zone-hour-day cells have fewer than 30 cleaned trips, so the map shows only reliable cells and uses medians.

5. Reflection and future work

Technical challenges. We designed the database before the cleaning pipeline existed. Our first *trips* table dropped the tip and fee columns, and the economics slice later needed them, so the schema was extended and merged into one file. The loader and the schema used different column names (*pickup_ts* against *pickup_datetime*) and the written API specification listed endpoints nobody had built; we fixed both by making the documentation follow the code and by adding tests that run the endpoints on an in-memory SQLite database, so they do not need MySQL. Pagination and top-route queries over 7.5 million rows were the slowest parts and still need measuring on the full database.

Team challenges. Each of us worked on a separate branch with a different part of the project, so interfaces (column names, JSON shapes, the heap's calling convention) drifted until we compared them side by side, and at least one merge had to be

undone. Agreeing the contract first (one schema file, one API specification) would have saved most of that time.

If this were a real product. (1) Deploy the API and a managed MySQL with the dashboard on static hosting. (2) Use a bounded heap of size K and keyset paging instead of OFFSET. (3) Load every month, with an incremental pipeline that appends only new files and rebuilds the statistics tables. (4) Precompute the heatmap instead of caching the first request. (5) Cluster neighbouring slow zones into corridors and compare against traffic or bus-lane data. (6) Add automated tests against a real MySQL in continuous integration.